React JS Notes: `map()`, `filter()`, and `reduce()`

Level: Beginner
Topics: JavaScript Array Methods, Rendering Lists in React, Filtering Data, Calculating Totals, and Method Chaining

---

1. What Are `map()`, `filter()`, and `reduce()`?

`map()`, `filter()`, and `reduce()` are JavaScript array methods commonly used in React applications to work with lists of data.

Method	Purpose	Returns
`map()`	Transforms every array item	A new array
`filter()`	Selects items that satisfy a condition	A new array
`reduce()`	Combines array items into one accumulated result	A value, such as a number, array, or object

Example array:

```
const numbers = [10, 20, 30, 40, 50];
```

Part 1: `map()`

2. What Is `map()`?

The `map()` method runs a function for every array element and returns a **new array** containing the results.

In React, `map()` is commonly used to display lists of elements.

Syntax

```
array.map((item) => {
    return result;
});
```

Example 1: JavaScript

```
const numbers = [10, 20, 30];
```

```
const result = numbers.map((num) => {
    return num * 2;
});

console.log(result);
```

Output:

```
[20, 40, 60]
```

Explanation:

- `10 * 2 = 20`
- `20 * 2 = 40`
- `30 * 2 = 60`

`map()` transforms every element and puts the results into a new array.

Example 2: Short Syntax

When an arrow function contains only one expression, we can omit the braces and `return`.

```
const numbers = [1, 2, 3, 4];

const result = numbers.map(num => num * 5);

console.log(result);
```

Output:

```
[5, 10, 15, 20]
```

Example 3: Using `map()` in React

```
function App() {
    const students = ["Amit", "Riya", "Jay"];

    return (
        <div>
            <h2>Student List</h2>

            {students.map((student) => (
                <p key={student}>{student}</p>
            ))}
        </div>
```

```
    );
}

export default App;
```

Output:

```
Student List

Amit
Riya
Jay
```

Explanation:

- `map()` processes each item in the `students` array.
- Each student name is displayed inside a paragraph.
- The `key` helps React identify each item in the rendered list.

Important: Use a stable, unique ID as the `key` whenever possible. In this example, the student names are used as keys because they are unique in the sample array. If names may repeat or change, use unique IDs instead.

Example 4: Displaying Product Cards

```
function App() {
    const products = [
        { id: 1, name: "Laptop", price: 50000 },
        { id: 2, name: "Mouse", price: 500 },
        { id: 3, name: "Keyboard", price: 1500 }
    ];

    return (
        <div>
            {products.map((product) => (
                <div key={product.id}>
                    <h3>{product.name}</h3>
                    <p>Price: Rs. {product.price}</p>
                </div>
            ))}
        </div>
    );
}

export default App;
```

Explanation: Each product object is used to generate one product card. The `product.id` value provides a stable key for each card.

When Do We Use `map()`?

- Displaying student lists.
- Creating product cards.
- Displaying tables from API data.
- Rendering navigation menu items.
- Showing multiple images or notifications.

Part 2: `filter()`

3. What Is `filter()`?

The `filter()` method selects array elements that satisfy a condition and returns a **new array** containing only those elements.

Syntax

```
array.filter((item) => {
    return condition;
});
```

The condition should return `true` or `false`.

- `true` → Keep the item.
- `false` → Exclude the item.

Example 1: Filter Even Numbers

```
const numbers = [1, 2, 3, 4, 5, 6];

const result = numbers.filter(num => num % 2 === 0);

console.log(result);
```

Output:

```
[2, 4, 6]
```

Explanation: The condition `num % 2 === 0` is true only for even numbers.

Example 2: Filter Marks

```
const marks = [25, 80, 45, 90, 30];

const passed = marks.filter(mark => mark >= 33);

console.log(passed);
```

Output:

```
[80, 45, 90]
```

Explanation: Only marks greater than or equal to `33` are selected.

Example 3: Using `filter()` in React

```
function App() {
    const students = [
        { id: 1, name: "Amit", passed: true },
        { id: 2, name: "Riya", passed: false },
        { id: 3, name: "Jay", passed: true }
    ];

    const passedStudents = students.filter(
        student => student.passed
    );

    return (
        <div>
            <h2>Passed Students</h2>

            {passedStudents.map(student => (
                <p key={student.id}>{student.name}</p>
            ))}
        </div>
    );
}

export default App;
```

Output:

```
Passed Students

Amit
Jay
```

Explanation:

- `filter()` selects students whose `passed` property is `true`.
- `map()` displays the selected students.
- `student.id` provides a stable key for each rendered paragraph.

Remember: `filter()` selects data; `map()` transforms or displays data.

Example 4: Filter Products by Price

```
const products = [
    { id: 1, name: "Laptop", price: 50000 },
    { id: 2, name: "Mouse", price: 500 },
    { id: 3, name: "Keyboard", price: 1500 }
];

const affordable = products.filter(
    product => product.price < 2000
);

console.log(affordable);
```

Output:

```
[
    { id: 2, name: "Mouse", price: 500 },
    { id: 3, name: "Keyboard", price: 1500 }
]
```

When Do We Use `filter()`?

- Finding students who passed.
- Showing products below a specific price.
- Searching for matching records.
- Displaying active users.
- Showing completed tasks.
- Filtering data received from an API.

Part 3: `reduce()`

4. What Is `reduce()`?

The `reduce()` method processes array elements and combines them into a single accumulated result.

The result can be:

- A number.
- A string.
- An array.
- An object.

A common beginner example is calculating the sum of numbers.

Syntax

```
array.reduce((accumulator, currentValue) => {
    return updatedAccumulator;
}, initialValue);
```

Meaning of Each Term

- **Accumulator:** The result collected so far.
- **Current value:** The array element currently being processed.
- **Initial value:** The starting value of the accumulator.

Example 1: Sum of Numbers

```
const numbers = [10, 20, 30, 40];

const total = numbers.reduce((sum, num) => {
    return sum + num;
}, 0);

console.log(total);
```

Output:

```
100
```

Dry Run

The initial value of `sum` is `0`.

Step	`sum` Before	`num`	Calculation	`sum` After
1	0	10	`0 + 10`	10
2	10	20	`10 + 20`	30
3	30	30	`30 + 30`	60
4	60	40	`60 + 40`	100

Final result: 100

Example 2: Short Syntax

```
const numbers = [1, 2, 3, 4, 5];

const total = numbers.reduce((sum, num) => sum + num, 0);

console.log(total);
```

Output:

```
15
```

Example 3: Calculate the Total Price in React

```
function App() {
    const products = [
        { id: 1, name: "Mouse", price: 500 },
        { id: 2, name: "Keyboard", price: 1500 },
        { id: 3, name: "Headphones", price: 2000 }
    ];

    const total = products.reduce(
        (sum, product) => sum + product.price,
        0
    );

    return (
        <div>
            <h2>Shopping Cart</h2>

            {products.map(product => (
                <p key={product.id}>
                    {product.name} - Rs. {product.price}
                </p>
            ))}

            <h3>Total: Rs. {total}</h3>
        </div>
    );
}

export default App;
```

Output:

```
Shopping Cart

Mouse - Rs. 500
Keyboard - Rs. 1500
Headphones - Rs. 2000

Total: Rs. 4000
```

Explanation:

- `map()` displays each product.
- `reduce()` calculates the total price.
- The initial value `0` ensures that the calculation starts at zero.

Example 4: Count Passed Students

```
const students = [
    { name: "Amit", passed: true },
    { name: "Riya", passed: false },
    { name: "Jay", passed: true }
];

const count = students.reduce(
    (total, student) => total + (student.passed ? 1 : 0),
    0
);

console.log(count);
```

Output:

```
2
```

Explanation:

- A student who passed adds `1` to the total.
- A student who did not pass adds `0`.
- The final count is `2`.

When Do We Use `reduce()`?

- Calculating shopping cart totals.
- Calculating total marks.
- Counting records that match a condition.
- Calculating averages after finding the total.
- Grouping data into an object.

- Summarizing API data.

Part 4: Difference Between `map()`, `filter()`, and `reduce()`

Feature	`map()`	`filter()`	`reduce()`
Purpose	Transform every item	Select matching items	Accumulate items into a result
Returns	A new array	A new array	An accumulated result
Callback commonly uses	Item	Item and condition	Accumulator and item
Example	Double every number	Keep even numbers	Calculate the sum

Quick Example

```
const numbers = [1, 2, 3, 4, 5];

const mapped = numbers.map(num => num * 2);
// [2, 4, 6, 8, 10]

const filtered = numbers.filter(num => num > 3);
// [4, 5]

const reduced = numbers.reduce((sum, num) => sum + num, 0);
// 15

console.log(mapped);
console.log(filtered);
console.log(reduced);
```

Part 5: Combining `filter()`, `map()`, and `reduce()`

These methods can be chained together to process data step by step.

Example: Calculate the Total Price of Affordable Products

Calculate the total price of products costing less than Rs. 2000.

```
const products = [
    { name: "Mouse", price: 500 },
```

```
    { name: "Keyboard", price: 1500 },
    { name: "Laptop", price: 50000 },
    { name: "Headphones", price: 1800 }
];

const total = products
    .filter(product => product.price < 2000)
    .map(product => product.price)
    .reduce((sum, price) => sum + price, 0);

console.log(total);
```

Output:

```
3800
```

Step-by-Step Explanation

Step 1: `filter()`

Select products costing less than Rs. 2000:

- Mouse — Rs. 500
- Keyboard — Rs. 1500
- Headphones — Rs. 1800

The laptop is excluded because it costs Rs. 50000.

Step 2: `map()`

Extract the prices:

```
[500, 1500, 1800]
```

Step 3: `reduce()`

Add the prices:

```
500 + 1500 + 1800 = 3800
```

Final total: Rs. 3800

---

6. Important Points to Remember

1. `map()`, `filter()`, and `reduce()` are JavaScript array methods, not React-specific methods.
2. React commonly uses `map()` to render lists.
3. `filter()` is useful for selecting which items should be displayed.
4. `reduce()` is useful for calculating totals and summaries.
5. These methods do not mutate the original array; they return a result based on its elements.
6. Always provide a unique, stable `key` when rendering a list in React.
7. Use `filter()` before `map()` when you need to display only matching items.
8. Use `reduce()` when you need to combine values into a result.
9. Providing an initial value to `reduce()`, such as `0` for addition, is generally a good practice.
10. These methods can be used without `useState`. Use state when data needs to change interactively and the UI should update in response.

---

7. Final Revision

- **`map()`** = Transform every item.
- **`filter()`** = Select matching items.
- **`reduce()`** = Combine items into a result.

Common React Use Cases

Requirement	Method
Display a list of products	`map()`
Display only affordable products	`filter()` + `map()`
Calculate the shopping cart total	`reduce()`
Display only passed students	`filter()` + `map()`
Count passed students	`reduce()`

---

8. Practice Tasks

Try solving these tasks without looking at the examples above.

Level 1: `map()`

1. Given `[2, 4, 6, 8]`, create a new array containing each number multiplied by `3`.
2. Given `["Amit", "Riya", "Jay"]`, create a new array containing each name in uppercase.
3. Create a React component that displays a list of five subjects using `map()`.

Level 2: `filter()`

4. Given `[12, 25, 30, 41, 50, 63]`, select only the even numbers.
5. Given `[25, 80, 45, 90, 30]`, select marks greater than or equal to `40`.
6. Given a product array, display only products priced below Rs. 1000.

Level 3: `reduce()`

7. Calculate the sum of `[10, 20, 30, 40, 50]`.
8. Calculate the total marks from an array of student marks.
9. Count how many students passed using an array of student objects.

Level 4: Combine the Methods

10. Given a product array, display products costing less than Rs. 5000 and calculate their total price.
11. Given a student array, display only students who passed and render each student's name.
12. Create a shopping cart component that displays product names, prices, and the final total.

Final Summary

Mastering `map()`, `filter()`, and `reduce()` helps you work confidently with arrays and dynamic data in React.

- Use `map()` to transform data or render lists.
- Use `filter()` to select data that matches a condition.
- Use `reduce()` to calculate totals and build summaries.